

UAV-Assisted Microservice Mobile Edge Computing Architecture: Addressing Post-Disaster Emergency Medical Rescue

Ji Li [✉], Qiang He [✉], Associate Member, IEEE, Xingwei Wang [✉], Ammar Hawbani [✉],
Keping Yu [✉], Senior Member, IEEE, Yuanguo Bi [✉], Member, IEEE, and Liang Zhao [✉]

Abstract—In post-disaster emergency medical rescue operations, rapidly establishing an adaptive and flexible edge computing (EC) network, balancing data offloading with energy consumption, and ensuring the stable operation of the network have become urgent priorities. To address these challenges, we proposed an unmanned aerial vehicle (UAV)-assisted microservice mobile edge computing (MEC) architecture. The architecture can be rapidly deployed to provide temporary network coverage and EC services in disaster-stricken areas. A transformer-based resource management (TBRM) approach is utilized to optimize data offloading efficiency and reduce energy consumption, thereby maximizing the service time of the architecture. To enhance the security and reliability of the architecture, four microservices are designed to manage the full UAV life-cycle, and UAV identity authentication is implemented through dual digital signature certificates. Large-scale simulation experiments have demonstrated the effectiveness of the architecture in complex rescue scenarios, providing strong technical support for post-disaster medical rescue efforts.

Index Terms—Unmanned aerial vehicle, mobile edge computing, transformer, Lyapunov optimization, post-disaster emergency medical rescue.

I. INTRODUCTION

IN recent years, the growing frequency of natural disasters, such as volcanic eruptions and earthquakes, has frequently resulted in large numbers of casualties and missing persons. Post-

disaster emergency medical rescue is crucial in the aftermath of such events. However, in the face of severe natural disasters, fundamental service infrastructures such as networks and power supplies are often severely damaged. Therefore, rapidly constructing an adaptive and flexible EC network [1], [2] in such an environment to support post-disaster emergency medical rescue [3], [4], [5], [6] remains a significant challenge. Fortunately, the flexibility and mobility of UAVs [7], [8], [9] offer an opportunity to achieve this goal. UAVs can move flexibly within an area and, when equipped with mobile base stations, can quickly adjust their positions to provide network service coverage. Additionally, UAVs can be equipped with MEC devices and sensor equipment, thereby providing essential computational capabilities for post-disaster emergency medical rescue.

Advances from heavyweight to lightweight convolutional neural network (CNN) architectures [10], [11], [12], [13], [14] have made it feasible to deploy deep learning models on resource-limited mobile edge devices. The Transformer [15], a foundational component of large models, incorporates a self-attention mechanism that can effortlessly handle input sequences of varying lengths and complexities without requiring predefined rules or boundaries. In the complex environment of post-disaster rescue, this capability enables rapid acquisition of critical information, accurate situation assessment, and dynamic adjustment of rescue strategies, thereby ensuring the most effective response in the shortest possible time and gaining valuable time for life-saving operations. Moreover, microservices have shown remarkable potential [16], [17], [18] in recent years through their deep integration with fields such as edge computing and the Internet of Things (IoT). It not only decomposes the system into independent services, ensuring high availability and fault isolation, but also facilitates rapid scaling and deployment based on demand. Especially in the context of emergency medical rescue, microservices can rapidly respond to and recover from unforeseen events. It is based on the immense potential of UAVs, Transformers, and microservices in post-disaster emergency medical rescue that we have conducted research on a UAV-assisted microservice MEC architecture. The main contributions can be summarized as follows:

- 1) We proposed a UAV-assisted microservice MEC architecture for post-disaster emergency medical rescue. The architecture can self-organize, automatically manage the entry and exit of UAVs, and provide long-term, secure,

Received 28 July 2024; revised 11 April 2025; accepted 25 April 2025. Date of publication 21 May 2025; date of current version 11 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62432003, Grant U22A2004, Grant 92267206, Grant 62202089, and Grant 72192832. (Corresponding author: Qiang He.)

Ji Li is with the College of Medicine and Biological Information Engineering, Northeastern University, Shenyang 110169, China (e-mail: jili13613@163.com).

Qiang He is with the College of Medicine and Biological Information Engineering, Northeastern University, Shenyang 110169, China and also with the College of Computer Science and Engineering, Northeastern University, Shenyang 110169, China (e-mail: heqiangcai@gmail.com).

Xingwei Wang and Yuanguo Bi are with the College of Computer Science and Engineering, Northeastern University, Shenyang 110169, China (e-mail: net2020zjq@163.com; biyuanguo@mail.neu.edu.cn).

Ammar Hawbani and Liang Zhao are with the School of Computer Science, Shenyang Aerospace University, Shenyang 110136, China (e-mail: anmande@ustc.edu.cn; lzhaol@sau.edu.cn).

Keping Yu is with the Graduate School of Science and Engineering, Hosei University, Tokyo 184-8584, Japan, and also with RIKEN Center for Advanced Intelligence Project, RIKEN, Tokyo 103-0027, Japan (e-mail: keping.yu@ieee.org).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TC.2025.3566913>, provided by the authors.

Digital Object Identifier 10.1109/TC.2025.3566913

stable, and efficient EC services to disaster-affected users (briefly noted as users).

- 2) We proposed a Transformer-based resource management method, which comprehensively considers data offloading, energy consumption, and response latency. This approach employs the Transformer-based channel allocation generator (TCAG) to generate offloading decisions and a dynamic resource optimization allocator (DROA) to execute various optimizations. A generator optimization strategy (GOS) is designed to update TCAG parameters, aiming to maximize the service time of the architecture.
- 3) We conducted a detailed analysis of the post-disaster emergency medical rescue scenario and subsequently designed four microservices. These four microservices encompass the full UAV lifecycle. In particular, dual digital signature certificates are utilized to ensure the security of UAVs registration, networking, and exit processes.
- 4) Unlike existing simulation validations, which only consider a small number of users and short service durations, we conducted extensive simulations with several times more users and longer service durations. This verified the effectiveness of the proposed method and demonstrated the feasibility of the proposed framework in real rescue environments.

The rest of this paper is organized as follows. Section II provides a review of previous research and literature related to the topic of the paper. Section III presents the system model and clearly defines the problem that the paper aims to solve. In Section IV, the paper presents the detailed design of the proposed method, namely the TBRM. Section V describes the experiments conducted to validate and evaluate the effectiveness of the proposed method. Section VI concludes the paper.

II. RELATED WORK

A. Edge Computing in Emergency Medical Rescue

In recent years, scholars have begun to apply EC technology to disaster rescue scenarios. Liu et al. [19] utilized the computational power of edge servers to perform real-time preprocessing and analysis of disaster images, thereby supporting emergency response and rescue operations. Mao et al. [20] designed a space-aerial-assisted hybrid cloud-edge computing framework, which utilizes UAVs and satellites to provide low-latency services for IoT devices, optimizing resource scheduling to minimize computational delays. The studies [21], [22] explored the feasibility of UAV-assisted intelligent EC from the perspective of supporting rescue personnel. Zhang et al. [23] studied the issues of fault tolerance and rapid response of computing services in disaster scenarios. They designed a fault-tolerant distributed task offloading scheme to minimize task execution time and system energy consumption using a multi-agent proximal policy optimization algorithm.

The aforementioned studies explore various aspects of using EC to assist in disaster relief in natural disaster environments from different perspectives. However, existing research still has limitations, particularly when considering disaster-affected users as the service target. It fails to adequately take into account the

sustainable operational capacity of emergency medical rescue in large-scale disaster scenarios.

B. Microservice-Based Edge Computing

Microservice architecture can decompose applications into multiple small, independent services, each of which can be developed, deployed, and scaled independently. Researchers have already integrated it with EC in various scenarios. The studies [24], [25], [26] explore microservice deployment and resource allocation from different perspectives, including microservice edge computing resource configuration methods, optimization of microservice instance selection, and deployment strategy optimization to reduce communication overhead based on deep Q-learning. Guo et al. [27] optimized microservice selection and scheduling in cloud-edge environments using the Markov decision process and the deep deterministic policy gradient algorithm, creating an experience pool to reduce user access latency. Tang et al. [28] proposed an adaptive microservice deployment method for IoT in MEC, using the Adam and weighted round-robin scheduling to minimize resource costs, meet delay constraints, and balance edge server workloads. Ray et al. [29] proposed a proactive mechanism based on reinforcement learning and learning automata for microservice deployment and migration in multi-access EC, tracking user mobility and server capacity to trigger migration and prevent overload.

From the above, it is clear that existing architectures combining microservices and EC primarily focus on the deployment, operation, and optimization of microservices. However, few studies have utilized microservice characteristics to design specialized applications in EC frameworks.

C. Resource Management Algorithms

Efficient management of computing resources and optimization of energy consumption are critical aspects in any MEC framework. Hoang et al. [30] proposed an online resource management framework based on dual-channel links. Their simulation, involving 8 users, 1 cellular base station, and 1 UAV, demonstrated the framework's ability to optimize average system power consumption. Zhang et al. [31] introduced a greedy-strategy-based orbital EC task allocation algorithm. It aims to utilize multiple access EC servers in low earth orbit satellite networks to serve ground users. Ding et al. [32] proposed a novel online edge learning offloading scheme for secure communication in UAV-assisted MEC, improving the performance of secure computing. Hasan et al. [33] explored the application of 6G networks in supporting autonomous driving and vehicle edge computing, emphasizing the role of computational offloading and resource management in reducing communication costs, protecting privacy, and reducing training processes. Zhang et al. [34] presented an action-constrained deep reinforcement learning approach for secure computation resource allocation, aimed at addressing the limited computing resources available on user devices.

Overall, resource management algorithms are numerous and varied, designed to address complex and changing scenarios.

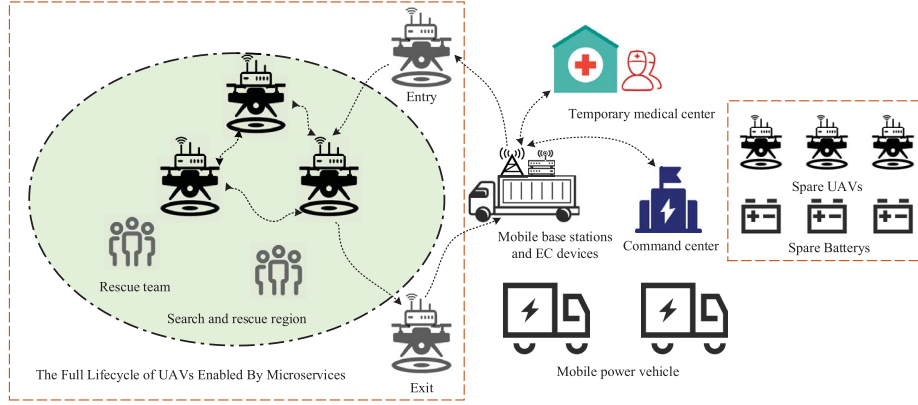


Fig. 1. UAV-assisted microservice MEC architecture.

However, for post-disaster rescue scenarios, due to their unique nature, no algorithm currently exists that minimizes system energy consumption by accounting for both UAV flight energy consumption and computing resource allocation.

III. SYSTEM MODEL

A. UAV-Assisted Microservice MEC Architecture

The overall structure of the UAV-assisted microservice MEC architecture is illustrated in Fig. 1. The UAV search queue equipped with MEC servers, along with the rescue team, constitute the primary forces in the search and rescue area. The UAV search queue provides temporary network and EC services in the search and rescue area, enabling users to contact rescue personnel, friends, and family for assistance. Furthermore, we propose four distinct microservices deployed on each UAV-MEC server involved in the rescue operation. These four microservices collaborate to support the full lifecycle of the UAVs. Behind the disaster area, trucks equipped with mobile base stations and edge devices provide network communication and real-time computing capabilities for the temporary medical center and the command center. They also receive real-time rescue data from the search area, and then employ advanced artificial intelligence models to analyze data and assist rescue coordination. Mobile power vehicles support the logistical operations by providing electricity for the entire area. Backup UAV units and battery packs remain on standby based on frontline rescue conditions to ensure the stability of the architecture. For ease of reference, key notations used in the paper are summarized in Table I.

B. Task Execution Model

In the UAV-assisted microservice MEC architecture, we consider that all requests from users can either be processed locally or offloaded to a UAV-MEC server. The decision between local computation or hybrid computation depends on a balance algorithm between latency and energy consumption. The scenario considered is depicted in Fig. 2: UAV-MEC servers are stationed at a height of 50 meters, providing services within a $2km \times 2km$ area using five UAVs. Users beyond the $2km \times 2km$ area can have UAV-MEC servers dispatched to their location

TABLE I
SUMMARY OF KEY NOTATIONS

Symbol	Description
S	The set of UAV-MEC servers
N	The set of users in the search area
f_n	The number of CPU cycles for user n
$Task_n$	The computational task generated by user n
m	The data size of $Task_n$
L_n	Required CPU cycles for user n to process 1 bit of data
$f_{s,n}$	CPU computation resources allocated by UAV-MEC server s to user n
L_s	Required CPU cycles for UAV-MEC server s to process 1 bit of data
$wtr_{s,n}$	The wireless transmission data rate between s and n
λ	The ratio of $Task_n$ offloaded locally
ξ_n	Effective switched capacitance of user n
ξ_s	Effective switched capacitance of UAV-MEC server s
p_n	The transmission power of user n 's device
ψ	Scaling factor
t_d	Hovering time
H	Relative hover/rise/fall height
M_g	The total weight of the UAV carrying various equipment
τ	Time slot length
v	UAV's velocity vector
t	Time slot
$\beta_{s,n}^t$	The link between user n and UAV-MEC server s in time slot t
$\alpha_{s,n}^t$	The ratio of bandwidth allocated to user n by UAV-MEC server s in time slot t
$r_{s,n}^t$	The volume of tasks offloaded by user n to UAV-MEC server s in time slot t
f_n^t	The local CPU frequency of user n in time slot t
$f_{s,n}^t$	The ratio of computational resources allocated to user n by UAV-MEC server s in time slot t
$Q_n^L(t)$	The task execution queue for user n in time slot t
D_n^t	The volume of tasks departing from the user n 's local queue in time slot t
I_n^t	The volume of tasks processed locally by user n in time slot t
A_n^t	The volume of newly arrived tasks generated by user n in time slot t
$Q_s^U(t)$	The task execution queue for UAV-MEC server s in time slot t
c_n^t	The volume of user n 's tasks processed by UAV-MEC server s in time slot t
$Z_n^L(t)$	The virtual task execution queue for user n in time slot t
$Z_s^U(t)$	The virtual task execution queue for UAV-MEC server s in time slot t
$h_{s,n}^t$	The channel power gain between user n and MEC server s in time slot t
W_s	The total bandwidth of UAV-MEC server s
N_0	The background noise density

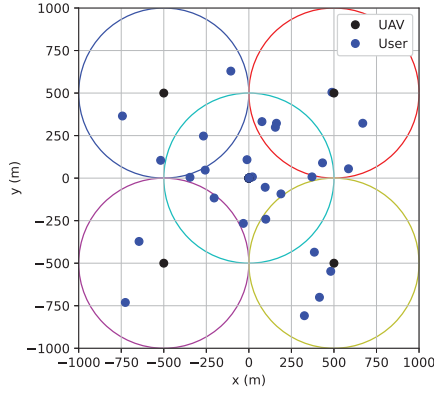


Fig. 2. Service Area of UAV-MEC servers.

through microservices control. Alternatively, a similar UAV-MEC server array can be deployed to provide the same computing offload services. In this scenario, a user may simultaneously fall within the coverage range of three UAV-MEC servers. However, in this architecture, we stipulate that a user can only offload tasks to one UAV-MEC server. This approach aims to minimize the complexity of the system, ensuring that the energy optimization and data offloading are performed within a fixed system environment. Specifically, we adopt the idea of load balancing to distribute users who are simultaneously covered by multiple UAV-MEC servers.

let $S = \{S_1, S_2, S_3, S_4, S_5\}$ represent the set of UAV-MEC servers in the initial state, $N_i = \{1, 2, 3, \dots, n\}$, $i \in \{1, 2, 3, 4, 5\}$ represent the set of users within the range of UAV-MEC server S_i . In the field of MEC, CPU cycles are a crucial metric for measuring the computational capacity of a device. For any user n , we denote f_n (cycles/s) as the number of CPU cycles of that user's device. The latest mobile CPU architectures utilize advanced dynamic frequency and voltage scaling technology, which can increase or decrease CPU cycles. Hence, $f_n \leq f_n^{\max}$. According to [35], for the computational task $Task_n$ generated by user n with a data size of m , the total execution time T^L for $Task_n$ to be fully executed locally can be expressed as:

$$T^L = \frac{mL_n}{f_n} \quad (1)$$

For example, in EC scenarios such as object detection or other machine learning tasks that process camera frames, execution time is typically proportional to data size. When the frame resolution matches the machine learning model input, higher resolution results in longer inference times due to increased computational workload. Similarly, let $f_{s,n}$ (cycles/s), $s \in S_i$, $n \in N_i$ represent the CPU computation resources allocated by UAV-MEC server s to user n , and $wtr_{s,n}$ is the wireless transmission data rate between UAV-MEC server s and user n . The total execution time T^S for user n to offload $Task_n$ entirely to UAV-MEC server s can be expressed as:

$$T^S = \frac{mL_s}{f_{s,n}} + \frac{m}{wtr_{s,n}} \quad (2)$$

In the designed architecture of this paper, multiple users can offload their tasks to the same UAV-MEC server, but one user

cannot offload tasks to multiple UAV-MEC servers simultaneously. Suppose the ratio of local computation and offloading for user n is λ ($0 \leq \lambda \leq 1$) and $1 - \lambda$, respectively. The total execution time T^{joint} can be expressed as:

$$T^{joint} = \frac{\lambda mL_n}{f_n} + \frac{(1 - \lambda)mL_s}{f_{s,n}} + \frac{(1 - \lambda)m}{wtr_{s,n}}. \quad (3)$$

C. Energy Consumption for Data Offloading

After a natural disaster, users cannot ensure the continuous power supply for their devices. Therefore, minimizing the energy consumption of user devices is essential for enabling timely and proactive rescue-seeking in energy-constrained scenarios. According to [36], the energy consumption per CPU cycle for user n is $\xi_n f_n^2$, where ξ_n is the effective switched capacitance, depending on the chip architecture. Hence, the energy consumption E^L for user n to process the computational $Task_n$ locally can be expressed as:

$$E^L = \xi_n mL_n f_n^2 \quad (4)$$

Similarly, the energy consumption E^{Total} for user n to offload the entire task data to UAV-MEC server s is:

$$E^{Total} = E^S + E^{L,Trans} \quad (5a)$$

$$E^S = \xi_s mL_s f_{s,n}^2 \quad (5b)$$

$$E^{L,Trans} = p_n \frac{m}{wtr_{s,n}} \quad (5c)$$

Next, consider the energy consumption for user n to offload the computing $Task_n$ in a hybrid manner. Suppose the ratio of local computation and offloading for user n is λ and $1 - \lambda$, respectively. The energy consumption E_{mix}^{Total} can be expressed as:

$$E_{mix}^{Total} = E_{mix}^S + E_{mix}^L + E_{mix}^{L,Trans} \quad (6a)$$

$$E_{mix}^S = (1 - \lambda) \xi_s mL_s f_{s,n}^2 \quad (6b)$$

$$E_{mix}^L = \lambda \xi_n mL_n f_n^2 \quad (6c)$$

$$E_{mix}^{L,Trans} = p_n \frac{(1 - \lambda)m}{wtr_{s,n}}. \quad (6d)$$

D. Energy Consumption for Flying

Another factor that impacts the overall service time of the architecture is the UAV flight energy consumption. Unlike [37] and [38], we take into account a energy consumption model that better reflects real-world application scenarios. Specifically, we draw on the validated conclusions of [39]. However, as the UAV model considered in [39] is lightweight, its direct application to rescue missions is limited. To overcome these limitations, we scaled up the flight energy consumption in [39] proportionally based on the UAV's weight to preserve the core properties and conclusions of the original energy consumption function. We stipulate that when the UAV departs from the command center base to reach a designated service area, the UAV will sequentially experience three flight states: vertical ascent, horizontal flight, and vertical descent. If the service time

in the current area ends, we only consider horizontal flight when the UAV moves from the current area to the next service area. When the battery is depleted and needs to return for recharging, the UAV will execute horizontal flight, and vertical descent. Next, we will introduce the energy consumption of each flight state using scaling factor ψ . Firstly, the energy consumption E_h for hovering is given by:

$$E_h = \int_0^t \psi t_d (a_1 H + a_2) \quad (7)$$

Where t_d is the hovering time, H is the hovering height, a_1 and a_2 are constants. The energy consumption E_{up} for vertical ascent is given by:

$$E_{up} = \psi(b_1 H^2 + b_2 H + b_3) \quad (8)$$

Where b_1 , b_2 and b_3 are constants. The energy consumption E_{down} for vertical descent is given by:

$$E_{down} = \psi(c_1 H^2 + c_2 H + c_3) \quad (9)$$

Where c_1 , c_2 and c_3 are constants. For horizontal flight energy consumption E_f , since the flight speed is not mentioned in [39], we consider using the simplified model from [38], which is represented as:

$$E_f = 0.5 M_g \tau \|v\|^2 \quad (10a)$$

$$v = \frac{\|loc(t+1) - loc(t)\|}{\tau} \quad (10b)$$

In the above equation, M_g represents the total weight of the UAV carrying various equipment, τ is the time slot length. v is the UAV's velocity vector, defined as the ratio of the distance increment to the time increment in the horizontal plane. $loc(t+1)$ represents the UAV's position coordinates in the horizontal plane at time $t+1$, and $loc(t)$ represents the UAV's position coordinates in the horizontal plane at time t (Please note that t here represents a specific time instant with zero duration).

E. Task Queue

In this architecture, two types of execution queues are considered: the local task execution queue Q_n^L and the UAV-MEC server task execution queue Q_s^U . The update process of $Q_n^L(t)$ and $Q_s^U(t)$ can be expressed as:

$$Q_n^L(t+1) = \max\{Q_n^L(t) - D_n^t, 0\} + A_n^t, n \in N_i, t \in T \quad (11)$$

$$Q_s^U(t+1) = \max\{Q_s^U(t) - c_{s,n}^t, 0\} + r_{s,n}^t, n \in N_i, s \in S_i, t \in T \quad (12)$$

Where $D_n^t \triangleq l_n^t + r_{s,n}^t$, $Q_n^L(0) = Q_s^U(0) = 0$. To ensure that users are not adversely affected by high latency, we define the average queue lengths for user n and UAV-MEC server s as $\bar{Q}_n^L$ and $\bar{Q}_s^U$, respectively.

$$\bar{Q}_n^L = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^T E[Q_n^L(t)] \leq Q_n^{th} \quad (13a)$$

$$\bar{Q}_s^U = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^T E[Q_s^U(t)] \leq Q_s^{th} \quad (13b)$$

In the above equations, Q_n^{th} and Q_s^{th} are the threshold values for the average queue lengths of user n and UAV-MEC server s , respectively.

F. Microservice Design

Given that the architecture must maintain continuous operation for extended periods in real-world application scenarios, the transmission of information and UAV scheduling become particularly crucial. We analyzed the post-disaster emergency medical rescue scenario in detail and proposed four distinct microservices to address its specific needs. Next, we will introduce the four microservices in terms of their functionality, key design points, and specific implementation.

1) UAV Registration and Authentication Service:

- **Functionality:** This service handles UAV registration requests, verifies the identities of newly added UAVs, and ensures that only authorized UAVs are admitted into the system.
- **Key Design Points:** The base command center is likened to a Root Certificate Authority (CA). Newly added UAVs are regarded as Subordinate CA. We designed a dual digital signature certificate system. This includes Digital Signature Registration Certificate (DSRC), Digital Signature Networking Certificate (DSNC), and Digital Signature Decommissioning Certificate (DSDC).
- **Specific Implementation:** The full lifecycle of a UAV includes three stages: registration, networking, and exit, as shown in Fig. 3. During the registration phase, the UAV first generates a DSRC using its private key. The Root CA decrypts this certificate using the UAV's public key. Only when Root CA confirms that the signature of the DSRC is from the UAV applying for registration, will the Root CA sign the certificate with its private key, while also recording the information of the UAV. Next, the UAV begins networking. In the networking phase, we adopt a "who verifies, who signs" principle to ensure security and traceability. The first registered UAV must obtain a DSNC from both the Root CA and itself. Subsequently, newly added UAVs can utilize their DSRC and self-signed DSNC to initiate networking requests with nearby UAVs that have already formed a network. Once the requested UAV verifies the authenticity of DSRC and DSNC, it will use its private key to add its digital signature to the applicant's DSNC, while simultaneously informing other UAVs of the networking information. When a UAV's battery is about to run out or it can no longer provide service, it needs to request exiting the network. Prior to this, it needs to sign a DSNC for the replacement UAV, allowing the new UAV to take over its duties and continue working. Additionally, it needs to synchronize relevant information with the replacement UAV. After the information synchronization is complete, the UAV that needs to exit uses its DSNC and self-signed DSDC to submit an exit request to the newly added UAV. After the request is approved, the new UAV adds its digital signature to the DSDC of the UAV

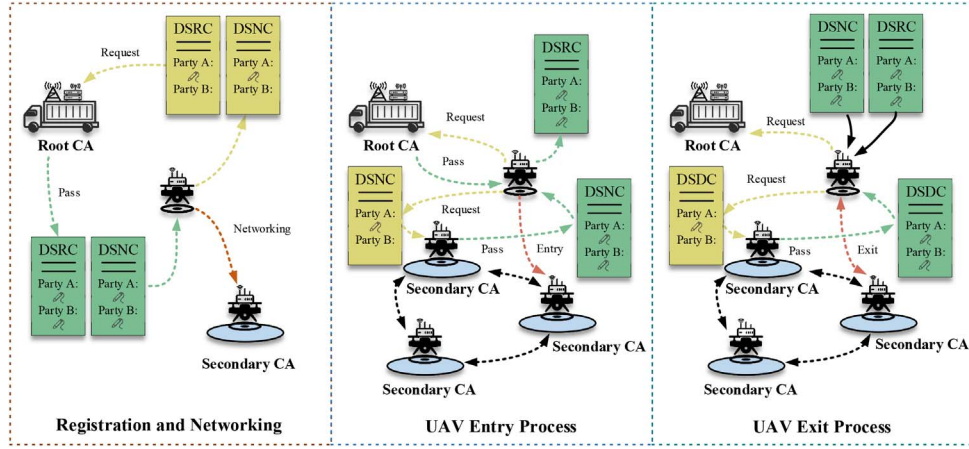


Fig. 3. UAV lifecycle diagram.

requesting to exit. Once the exiting UAV returns to the base, it completes secure decommissioning by validating its DSDC.

2) UAV States Management Service:

- **Functionality:** The real-time collection, storage, and updating of UAV status information, such as location, battery level, health status, provide data support for other services.
- **Key Design Points:** Utilize the lightweight message queue framework Kafka to achieve real-time collection of UAV status information, providing comprehensive support for the architecture with relevant UAV data.
- **Specific Implementation:** Since the MEC architecture decomposes optimization tasks into deterministic problems for each time slot, the UAV status management service can update and collect the UAV status information after the optimization for each time slot is completed.

3) UAV Scheduling Service:

- **Functionality:** Based on task requirements and UAV status, a scheduling plan is formulated to achieve automatic allocation and collaboration among UAVs.
- **Key Design Points:** This service needs to ensure that the energy consumption error during UAV scheduling remains within a certain range and maximizes the service time of UAVs.
- **Specific Implementation:** Under the proposed architecture, the UAV scheduling problem focuses mainly on two aspects. One aspect involves the entry and exit of UAVs, while the other concerns the overall movement of the UAVs' service areas. The latter only requires specifying the target position and following a predetermined flight route. However, the former requires establishing entry and exit rules, along with energy calculations, to fulfill specified requirements. We utilize long-term CPU utilization (For 100 consecutive time slots, the average CPU utilization exceeds 90%) as an evaluation metric to determine whether new UAVs need to be deployed. It should be noted that newly added UAVs may exit the system only after completing service in the current area

or when their power is nearly depleted. This is to avoid energy consumption caused by repeated joining and exiting of UAVs. The newly added UAVs will fly to a location near the UAV that made the request to maintain the stability of the rescue network. Simultaneously, we calculate the energy consumption for returning to the base using the UAV status indicators provided by the UAV status management service, which defines the minimum battery level indicator for formulating UAV exit plans.

4) Communication With Edge Computing Service:

- **Functionality:** Ensure communication between UAVs to support data transmission and collaborative work, while leveraging the computational capabilities of UAV-MEC servers to provide EC services for disaster-affected areas.
- **Key Design Points:** Ensure low-latency and reliable communication between UAVs, while also embedding essential computational resource management within this service.
- **Specific Implementation:** We employ the lightweight constrained application protocol (CoAP). CoAP is selected because it requires minimal network bandwidth and device resources, and its non-persistent connection mode reduces energy consumption. Additionally, CoAP provides a secure transmission layer based on datagram transport layer security, ensuring the confidentiality and integrity of data during transmission. The computational resource management is implemented using the TBRM method discussed in Section IV.

G. Cost Optimization Problem

The system's energy consumption E_{sys} comprises local execution energy consumption by the user, UAV-MEC server execution energy consumption, data transmission energy consumption, and UAV flight energy consumption. This can be formulated as:

$$E_{\text{sys}} = E_{\text{mix}}^{\text{Total}} + E_s^{\text{Total}} \quad (14)$$

$E_s^{\text{Total}} (s \in S)$ represents the flight energy consumption of all UAVs participating in the rescue operation. The factors affecting

energy consumption (i.e., the variables that need to be optimized) are summarized as follows:

- 1) The link between users and UAV-MEC servers in time slot t .
- 2) The bandwidth resources allocated by UAV-MEC servers to each user in time slot t .
- 3) The volume of tasks offloaded by users to the UAV-MEC servers in time slot t .
- 4) The local CPU frequency of users in time slot t .
- 5) The computational resources allocated by UAV-MEC servers to each user in time slot t .

We denote the aforementioned factors as β_s^t , α_s^t , r_s^t , f_n^t , and f_s^t , respectively. Let $X^t = \{\beta_s^t, \alpha_s^t, r_s^t, f_n^t, f_s^t\}$ represent the combination of all optimization vectors of user n and UAV-MEC server s in time slot t . Let $X = \{X^t\}_{t \in T}$ represent the combination of all optimization variables over time. Our objective is to minimize the long-term average system energy consumption to maximize the system's service time and the users' connectivity duration. Therefore, the optimization problem can be formulated as follows:

$$P1 \quad \min_X \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} E[E_{\text{sys}}(t)] \quad (15)$$

$$s.t. \quad \beta_s^t = \{\beta_{s,1}^t, \beta_{s,2}^t, \dots, \beta_{s,n}^t\}, \beta_{s,n}^t = \{0, 1\}, \\ n \in N_i, s \in S_i, t \in T \quad (16a)$$

$$\alpha_s^t = \{\alpha_{s,1}^t, \alpha_{s,2}^t, \dots, \alpha_{s,n}^t\}, 0 \leq \alpha_{s,n}^t, \sum_{n=1}^{N_i} \alpha_{s,n}^t \leq 1, \\ n \in N_i, s \in S_i, t \in T \quad (16b)$$

$$r_s^t = \{r_{s,1}^t, r_{s,2}^t, \dots, r_{s,n}^t\}, 0 \leq r_{s,n}^t \leq r_{s,n}^{\max}, \\ n \in N_i, s \in S_i, t \in T \quad (16c)$$

$$0 \leq f_n^t \leq f_n^{\max}, n \in N_i, t \in T \quad (16d)$$

$$\sum_{n=1}^{N_i} f_{s,n}^t \leq f_s^{\max}, s \in S_i, n \in N_i, t \in T \quad (16e)$$

$$(13a) \text{ and } (13b) \quad (16f)$$

It is important to note that the above constraints ensure that the user set N_i is within the coverage area of the UAV-MEC server S_i . Constraint (16a) limits whether the user's computational tasks require the involvement of the UAV-MEC server. Constraint (16b) stipulates that the transmission bandwidth allocated by the UAV-MEC server to each user cannot exceed the maximum resource limit of the UAV-MEC server. Constraint (16c) limits the maximum offloading volume by user n to UAV-MEC server s in time slot t . Constraint (16d) indicates that the change in the user's frequency should remain within the maximum allowable range. Constraint (16e) restricts the computational resources allocated by the UAV-MEC server to the users within its coverage area, ensuring it does not exceed the maximum permissible frequency. (13a) and (13b) are delay constraints.

To address the optimization problem $P1$, it is essential to analyze its time complexity. The number of optimization variables is crucial in determining the time complexity. According to the model's setup, in each time slot t , the variables β_s^t , α_s^t , r_s^t , f_n^t and f_s^t must be

optimized. Since users can only offload tasks to a UAV-MEC server, it follows that for each time slot, the number of variables for each type is $O(N)$. Considering T time slots, the total number of variables accumulates to $O(T \times N)$. In this case, the typical time complexity for a linear programming solver is between $O((T \times N)^3)$ and $O((T \times N)^{3.5})$, which is often impractical. On the other hand, dynamic programming methods tend to exhibit a drastic increase in time complexity as the number of state transitions grows, potentially leading to a form like $O(T \times N \times \text{state transitions})$, which could impose a significant computational burden. In contrast, the lyapunov method offers a more efficient solution. By decomposing the global optimization problem into a series of local optimization subproblems, this approach simplifies the optimization process at each time step while avoiding the complexity of directly tackling the global optimization problem.

H. Lyapunov-Guided Problem Transformation

We employ the lyapunov framework to transform the above optimization problem $P1$ into a deterministic problem for each time slot. To satisfy constraints (13a) and (13b), we introduce two virtual queues:

$$Z_n^L(t+1) = \max\{Z_n^L(t) + Q_n^L(t+1) - Q_n^{\text{th}}, 0\} \quad (17a)$$

$$Z_s^U(t+1) = \max\{Z_s^U(t) + Q_s^U(t+1) - Q_s^{\text{th}}, 0\} \quad (17b)$$

Specifically, $Z_n^L(0) = Z_s^U(0)$. Additionally, we define the system's state in time slot t as $\Theta(t) \triangleq \{Q_n^L, Q_s^U, Z_n^L, Z_s^U\}_{n \in N_i, s \in S_i}$. The lyapunov function is defined as follows:

$$L(\Theta(t)) \triangleq \frac{1}{2} \sum_{n \in N_i, s \in S_i} [Q_n^L(t)^2 + Q_s^U(t)^2 + Z_n^L(t)^2 + Z_s^U(t)^2] \quad (18)$$

At this point, the lyapunov drift function can be expressed as:

$$\Delta(\Theta(t)) \triangleq E[L(\Theta(t+1)) - L(\Theta(t)) | \Theta(t)] \quad (19)$$

To minimize the long-term average energy consumption while ensuring queue stability constraints, we define the lyapunov-drift-plus-penalty as:

$$\Delta_V(\Theta(t)) \triangleq \Delta(\Theta(t)) + VE[E_{\text{sys}}(t) | \Theta(t)] \quad (20)$$

Where V is a positive parameter that controls the trade-off between system energy consumption and average queue delay. We provide the upper bound of $\Delta_V(\Theta(t))$ in Theorem 1, which is very important for transforming the multistage problem $P1$ into the deterministic problem of each time slot. With the support of Theorem 1, we can use the technique of opportunistic expectation minimization to transform problem $P1$ into a deterministic problem solvable for each time slot.

Theorem 1: The Lyapunov-drift-plus-penalty $\Delta_V(\Theta(t))$ is bounded as

$$\Delta_V(\Theta(t)) \leq \hat{B} - \sum_{n \in N_i} E[(Q_n^L(t) + Z_n^L(t))(D_n^t - A_n^t) | \Theta(t)] \\ - \sum_{n \in N_i, s \in S_i} E[(Q_s^U(t) + Z_s^U(t))(c_n^t - r_{s,n}^t) | \Theta(t)] \\ + VE[E_{\text{sys}}(t) | \Theta(t)] \quad (21)$$

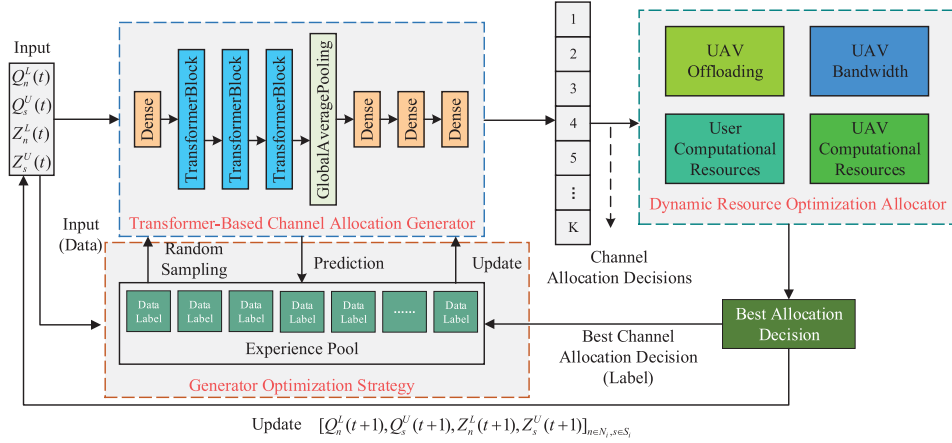


Fig. 4. Transformer-based resource management.

Specifically, by removing the constant term in Theorem 1 and minimizing the upper bound of $\Delta_V(\Theta(t))$, the deterministic problem for each time slot is represented as follows:

$$\begin{aligned} P2 \quad & \min_{X'} G(X') \\ s.t. \quad & (16a)-(16f) \end{aligned} \quad (22)$$

The objective function $G(X')$ can be expressed as follows:

$$\begin{aligned} G(X') = & - \sum_{n \in N_i} [(Q_n^L(t) + Z_n^L(t))(I_n^t + r_{s,n}^t)] \\ & - \sum_{n \in N_i, s \in S_i} [(Q_n^U(t) + Z_n^U(t))(c_n^t - r_{s,n}^t)] \\ & + V(E_{sys}(t)) \end{aligned} \quad (23)$$

IV. TRANSFORMER-BASED RESOURCE MANAGEMENT

We propose the TBRM method to solve the optimization problem $P2$ in each time slot. This method consists of three components, as shown in Fig. 4.

- 1) Transformer-Based Channel Allocation Generator
- 2) Dynamic Resource Optimization Allocator
- 3) Generator Optimization Strategy

The input of the TCAG is $[Q_n^L(t), Q_n^U(t), Z_n^L(t), Z_n^U(t)]_{n \in N_i, s \in S_i}$. The output can be represented as $\{\tilde{x}_{m,s}^t\}_{m=1}^k$, which denotes the k channel allocation decisions for the current server s . Once the temporary channel allocations are determined, $P2$ can be decomposed into several subproblems with independent objectives and constraints. Subsequently, the DROA conducts detailed optimization on the k generated potential channel allocation decisions, determining the optimal offloading volume, bandwidth allocation, and other parameters under the current channel allocation decision. By selecting the sub-optimal allocation decision, user tasks are processed through offloading. Next, the GOS uses the sub-optimal channel allocation decision as pseudo-labels to update the TCAG. In the following sections, we will provide a detailed introduction to each component.

A. Transformer-Based Channel Allocation Generator

The TCAG consists of transformer multi-head attention layers, normalization operations, regularization, fully connected layers,

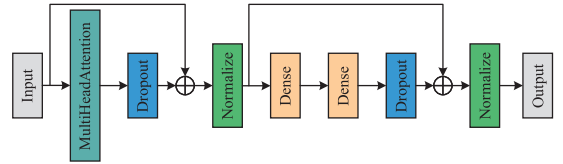


Fig. 5. Structure of the TransformerBlock.

global average pooling layer, and sigmoid activation functions. The overall network structure is shown as the TCAG in Fig. 4. The TCAG first uses fully connected layers to linearly combine the input features using its weights and biases, which helps to capture interactions and combinations between features. By introducing the ReLU activation function, the network model can learn complex nonlinear relationships. Next, the TCAG adds TransformerBlock layers (the structure of a TransformerBlock is shown in Fig. 5.) in a loop, enabling the model to capture long-term dependencies in the input sequence. Each TransformerBlock includes a multi-head attention mechanism, residual connections, normalization, and dropout layers. The multi-head attention mechanism splits the input into multiple “heads” to process in parallel, with each head learning different parts of the input, thereby enhancing the model’s ability to capture information. The residual connections introduce an identity mapping that helps mitigate the vanishing or exploding gradient problem. The normalization layer stabilizes the training process and accelerates convergence, while the dropout layer prevents overfitting by randomly dropping out some neurons’ outputs. Afterward, the TCAG uses a global average pooling layer to average the feature sequence along the sequence dimension, reducing the number of parameters and computational load while retaining the most important features. Finally, the TCAG employs successive fully connected layers to map the extracted features to the final output classes. After generating the channel allocation probability decisions for each user using TCAG, the decisions are ranked according to these probabilities to obtain one channel allocation decision. To obtain the remaining $k - 1$ channel allocation decisions, Gaussian noise is added to the channel allocation probabilities followed by the same selection process. The mean of the gaussian noise is 0, the variance is $\sigma_n^2 \mathbf{I}$, and $\mathbf{I}$ represents the identity matrix.

B. Dynamic Resource Optimization Allocator

After obtaining the temporary allocation decisions predicted by the TCAG, we decompose $P2$ into four sub-problems. Using a model-based approach, we can precisely determine the sub-optimal solution $y^t = [\alpha_s^t, r_s^t, f_s^t, f_n^t]_{n \in N_i, s \in S_i}$ in time slot t .

1) *Optimize Offload and Bandwidth Allocation*: For a given channel allocation, the optimization variables for the UAV-MEC servers include bandwidth allocation $\alpha_s^t \triangleq \{\alpha_{s,n}^t | n \in N_i, s \in S_i\}$ and offloading volume $r_s^t \triangleq \{r_{s,n}^t | n \in N_i, s \in S_i\}$. Let $x^t \triangleq \{\alpha_s^t, r_s^t\}$ represent this set of variables. The optimization decision with x^t as the independent variable can then be represented as $P2.1$:

$P2.1$

$$\min_{x^t} - \sum_{n \in N_i, s \in S_i} [(Q_n^L(t) + Z_n^L(t) - Q_s^U(t) - Z_s^U(t))r_{s,n}^t] + V \sum_{n \in N_i} E_{mix}^{L,Trans} \quad (24)$$

$E_{mix}^{L,Trans}$ can be calculated using the Shannon-Hartley theorem:

$$E_{mix}^{L,Trans}(t) = \sum_{n \in N_i, s \in S_i} (2^{\frac{r_{s,n}^t}{W_s \alpha_{s,n}^t \tau}} - 1) \frac{N_0 W_s}{h_{s,n}^t} \quad (25)$$

To ensure the differentiability of equation (24), we artificially set a lower bound $\alpha_{s,n}^t$, denoted as $\varepsilon \leq \alpha_{s,n}^t$. As long as ε is sufficiently small, the result will approximate the original equation (24) closely. Under this condition, we can rewrite $P2.1$ and specify its constraints as follows:

$$P2.2 \quad \min_{x^t} - \sum_{n \in N_i} [(Q_n^L(t) + Z_n^L(t) - Q_s^U(t) - Z_s^U(t))r_{s,n}^t] + V \sum_{n \in N_i, s \in S_i} (2^{\frac{r_{s,n}^t}{W_s \alpha_{s,n}^t \tau}} - 1) \frac{N_0 W_s}{h_{s,n}^t} \quad (26)$$

$$s.t. \quad \varepsilon \leq \alpha_{s,n}^t, \quad \sum_{n \in N_i, s \in S_i} \alpha_{s,n}^t \leq 1 \quad (27a)$$

$$0 \leq r_{s,n}^t \leq \min\{Q_n^L(t), r_{s,n}^{t,max}\} \quad (27b)$$

Where $r_{s,n}^{t,max} = W_s \alpha_{s,n}^t \tau \log_2 \left(1 + \frac{p_{s,n}^{max} h_{s,n}^t}{N_0 W_s}\right)$ represents the upper bound of $r_{s,n}^t$, which corresponds to the scenario with maximum transmission power $p_{s,n}^{max}$. For users without an established channel allocation, $r_{s,n}^{t,max} = 0$ and $\alpha_{s,n}^t = 0$. To solve $P2.2$, we employ the gauss-seidel method to alternately optimize the offloading volume and bandwidth allocation. In each iteration, the offloading volume is determined in a closed-form solution, while the bandwidth allocation is determined using the lagrangian method. This alternating approach ensures convergence to the optimal solution because $P2.2$ is convex and the feasible region is the cartesian product of r_s^t and α_s^t .

a) *Optimal offloading volume*: For a feasible bandwidth allocation, the optimal offloading volume for each user can be determined by solving the following problem:

$$P2.2.1 \quad \text{minimize}_{r_s^t} (26) \quad \text{subject to} (27b) \quad (28)$$

The optimal solution to $P2.2.1$ is either a stationary point of (26) or one of its boundary points. Specifically, when $Q_n^L(t) + Z_n^L(t) \leq Q_s^U(t) + Z_s^U(t)$, the optimal solution for $r_{s,n}^t$ is 0. Otherwise, $(r_{s,n}^t)^\dagger = \max\{\min\{\hat{r}_{s,n}^t, r_{s,n}^{t,max}\}, 0\}$. The superscript $\dagger$ denotes the optimal solution, which is the same in the following context. Here, $\hat{r}_{s,n}^t = W_s \alpha_{s,n}^t \tau \times \log_2 \left(\frac{Q_n^L(t) + Z_n^L(t) - Q_s^U(t) + Z_s^U(t)}{V \ln(2) N_0} \right)$.

b) *Optimal bandwidth allocation*: For feasible offloading optimization decisions r_s^t , the optimal bandwidth allocation can be obtained by solving the following optimization problem:

$$P2.2.2 \quad \min_{\alpha_s^t} \sum_{n \in N_i, s \in S_i} E_{mix}^{L,Trans}(t) \quad (29)$$

$$s.t. \quad \varepsilon \leq \alpha_{s,n}^t, \quad \sum_{n \in N_i, s \in S_i} \alpha_{s,n}^t \leq 1 \quad (30a)$$

$$E_{mix}^{L,Trans}(t) \leq E_{mix}^{L,Trans,max} \quad (30b)$$

The equation (30b) is equivalent to $\alpha_{s,n}^t \geq \alpha_{s,n}^{t,min}$.

$$\alpha_{s,n}^{t,min} \triangleq \frac{r_{s,n}^t}{W_s \tau \log_2 \left(1 + \frac{E_{mix}^{L,Trans,max} h_{s,n}^t}{N_0 W_s}\right)} \quad (31)$$

From this, it can be concluded that the partial lagrange function related to the above problems can be expressed as:

$$L(\alpha_{s,n}^t, \lambda_s^t) \triangleq \sum_{n \in N_i, s \in S_i} \left(2^{\frac{r_{s,n}^t}{W_s \alpha_{s,n}^t \tau}} - 1\right) \frac{N_0 W_s}{h_{s,n}^t} + \lambda_s^t \left(\sum_{n \in N_i, s \in S_i} \alpha_{s,n}^t - 1 \right) \quad (32)$$

$\lambda_s^t \geq 0$ is the lagrange multiplier associated with constraint $\sum_{n \in N_i, s \in S_i} \alpha_{s,n}^t \leq 1$. Based on the karush-kuhn-tucker conditions, $(\alpha_s^t)^\dagger$ and $(\lambda_s^t)^\dagger$ should satisfy the following set of equations.

$$\begin{cases} \sum_{n \in N_i, s \in S_i} (\alpha_{s,n}^t)^\dagger = 1 \\ (\alpha_{s,n}^t)^\dagger = \max\{\hat{\varepsilon}, R_{s,n}((\lambda_s^t)^\dagger)\}, n \in N_i, s \in S_i \end{cases} \quad (33)$$

Where $\hat{\varepsilon} = \max\{\varepsilon, \alpha_{s,n}^{t,min}\}$, and $R_{s,n}(\lambda_s^t)$ represents the root of $\frac{\partial L(\alpha_{s,n}^t, \lambda_s^t)}{\partial \alpha_{s,n}^t} = 0$. The following proposition provides a closed-form expression for $R_{s,n}(\lambda_s^t)$.

Proposition 1: Given $\lambda_s^t \geq 0$, the root of $\frac{\partial L(\alpha_{s,n}^t, \lambda_s^t)}{\partial \alpha_{s,n}^t} = 0$ is non-negative and unique.

$$R_{s,n}(\lambda_s^t) = \frac{r_{s,n}^t \ln(2)}{2 W_s \tau \omega \left(\frac{r_{s,n}^t \ln(2)}{2 W_s \tau} \sqrt{\frac{\lambda_s^t h_{s,n}^t W_s \tau}{\sigma_s^2 r_{s,n}^t \ln(2)}} \right)} \quad (34)$$

Where $\omega(\cdot)$ represents the lambert-w function. Note that $\frac{\partial L(\alpha_{s,n}^t, \lambda_s^t)}{\partial \alpha_{s,n}^t}$ is a monotonic function of $\alpha_{s,n}^t$, hence $(\lambda_s^t)^\dagger$ can be found using a bisection method within the interval

$[\lambda_s^L(t); \lambda_s^U(t)]$. The selected $\lambda_s^L(t)$ and $\lambda_s^U(t)$ ensure that $\sum_{n \in N_i, s \in S_i} \max\{\hat{e}, R_{s,n}(\lambda_s^L(t))\} > 1$ and $\sum_{n \in N_i, s \in S_i} \max\{\hat{e}, R_{s,n}(\lambda_s^U(t))\} < 1$. The calculation method for $\lambda_s^L(t)$ and $\lambda_s^U(t)$ is shown below.

$$\begin{cases} \lambda_s^L(t) = \max_{n \in N_i, s \in S_i} \frac{-\partial E_{mix}^{L,Trans}(t)}{\partial \alpha_{s,n}^L} \Big|_{\alpha_{s,n}^L=1} \\ \lambda_s^U(t) = \max_{n \in N_i, s \in S_i} \frac{-\partial E_{mix}^{L,Trans}(t)}{\partial \alpha_{s,n}^U} \Big|_{\alpha_{s,n}^U=\hat{e}} \end{cases} \quad (35)$$

When $\left| \sum_{n \in N_i, s \in S_i} \max\{\hat{e}, R_{s,n}(\lambda_s^U(t))\} - 1 \right| < \eta$, the search process for $(\lambda_s^U)^\dagger$ terminates, with η representing the search precision.

2) *Optimization on User's Local Computation*: Given the offloading volume for each user, the local computation resource allocation problem can be decomposed into independent sub-problems for each user.

$$P2.3 \quad \min_{f_n^t} \sum_{n \in N_i} (-Q_n^L(t) \tau f_n^t L_n^{-1} + V \zeta_n (f_n^t)^3) \quad (36)$$

$$s.t. \quad 0 \leq f_n^t \leq f_n^{\max}, n \in N_i, t \in T \quad (37a)$$

$$l_n^t \leq Q_n^L(t) - (r_{s,n}^t)^\dagger, n \in N_i, s \in S_i \quad (37b)$$

It's evident that equation (36) represents a convex problem with all constraints being linear. Specifically, the optimal solution to problem P2.3 either lies at a stationary point of the objective function (36) or at a boundary point of the feasible region $(f_n^t)^\dagger = \min\left\{F_n, \sqrt{\frac{Q_n^L(t)\tau}{3\zeta_n V L_n}}\right\}, n \in N_i$, where $F_n = \min\{f_n^{\max}, (Q_n^L(t) - (r_{s,n}^t)^\dagger) L_n / \tau\}$.

3) *Optimization on UAV-MEC Computational Resource Scheduling*: Similar to the optimization of local computation resources for each user, the optimization of UAV-MEC computing frequency $f_s^t(t)$ can be obtained by solving problem P2.4.

$$P2.4 \quad \min_{f_s^t(t)} \sum_{n \in N_i, s \in S_i} -Q_s^U(t) \tau f_{s,n}^t L_s^{-1} + \sum_{n \in N_i, s \in S_i} V \zeta_s (f_{s,n}^t)^3 \quad (38)$$

$$s.t. \quad 0 \leq f_{s,n}^t \leq f_s^{\max}, n \in N_i, s \in S_i \quad (39a)$$

$$\tau f_{s,n}^t L_s^{-1} \leq Q_s^U(t), n \in N_i, s \in S_i \quad (39b)$$

The solution of P2.4 is similar to that of P2.3. Specifically, the optimal solution of problem P2.4 is either at the stationary point of the objective function (38) or at some boundary point of $(f_{s,n}^t)^\dagger = \min\left\{f_{s,n}^{\max}, \sqrt{\frac{Q_s^U(t)\tau}{3\zeta_s V L_s}}\right\}, n \in N_i, s \in S_i$, where $f_{s,n}^{\max} = \min\{f_s^{\max}, Q_s^U(t) L_s / \tau\}$.

C. Generator Optimization Strategy

After optimization through DROA, the sub-optimal decision is generated. At this time, GOS treats the optimized offloading

decisions as labels and the TCAG-generated sub-optimal channel allocations as input data. The binary cross-entropy loss function is employed to calculate the loss, providing a basis for analyzing TCAG's predictive capabilities. Additionally, both the labels and data are stored in a fixed-size experience pool, providing a solid foundation for model updates, iterations, and optimizations. Specifically, the system is configured to randomly sample a portion of data from the experience pool after a fixed number of iterations and apply normalization to the sampled data. This batch of data is then utilized to train the TCAG and update its model parameters, improving the accuracy of the generator model in subsequent predictions.

D. Computational Complexity Analysis

The computation mainly comes from DROA, as this component has to examine k TCAG-generated potential channel allocation decisions in each time slot and select the best one among them.

The complexity of joint optimization on offloading volume and bandwidth allocation: After given the feasible offloading volume, each UAV-MEC server needs $O(\log_2(N_i/\eta))$ iterations to find the optimal bandwidth allocation. Given a feasible bandwidth allocation, the optimization of the offload volume per user can be obtained in closed form. Thus, the complexity is in $O(N_i)$ considering N_i users. Finally, the complexity of gauss-seidel joint optimization is $O(I_{\max}(N_i + \log_2(N_i/\eta)))$ ($I_{\max}$ is the maximum number of iterations).

The complexity of optimization on the computation of the user and the UAV-MEC server: Since solutions to P2.3 and P2.4 can both be obtained in closed forms, the optimization is with complexity $O(1)$ for each user.

In summary, the complexity of each UAV-MEC server evaluating a channel allocation decision is $O(I_{\max}(N_i + \log_2(N_i/\eta)))$. Since DROA examines k potential decisions at each time slot, the overall computational complexity of the proposed method is $O(k I_{\max}(\sum_{i=1}^s N_i + \sum_{i=1}^s \log_2(N_i/\eta)))$.

V. SIMULATIONS AND PERFORMANCE EVALUATIONS

A. Parameter Settings

We set the UAV takeoff position at a distance of 2 km from the center of the service area. The maximum number of connections for each UAV-MEC server is set to 2. For all users, their tasks follow a poisson distribution, denoted as $E(A_n^t) = 15\text{kb}$; $N = 40$; $\psi = 6.03125$; $a_1 = 13.0397$; $a_2 = 196.849$; $b_1 = -16.9396$; $b_2 = 216.6944$; $b_3 = -157.9473$; $c_1 = 4.6817$; $c_2 = -11.9708$; $c_3 = 135.3118$; $M_g = 9.65\text{kg}$; $\tau = 10\text{ms}$; $f_s^{\max} = 4\text{GHz}$; $W_s = 4\text{GHz}$; $f_n^{\max} = 1\text{GHz}$; $N_0 = -174\text{dBm/Hz}$; $L_n = L_s = 737.5\text{cycles/bit}$; $Q_n^h = Q_s^h = 45\text{kb}$; $k = 16$; $V = 1 \times 10^9$; $\zeta_n = \zeta_s = 10^{-28}\text{Ws}^3/\text{cycle}^3$.

B. Comparison Method Setup

To demonstrate the effectiveness of our proposed method, we compare it with four different methods. These methods are as follows:

- 1) Deep Reinforcement Learning (DRL): Derived from recent study [30] that publicly released simulation code.

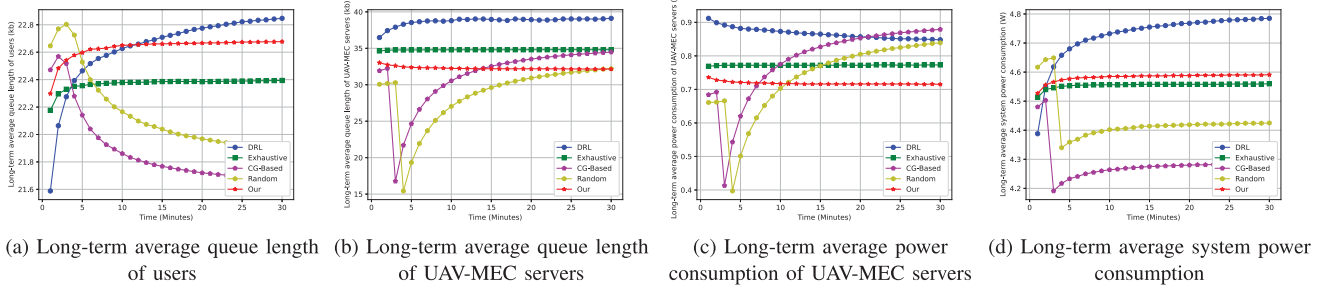


Fig. 6. Comparison of different methods under scenario I.

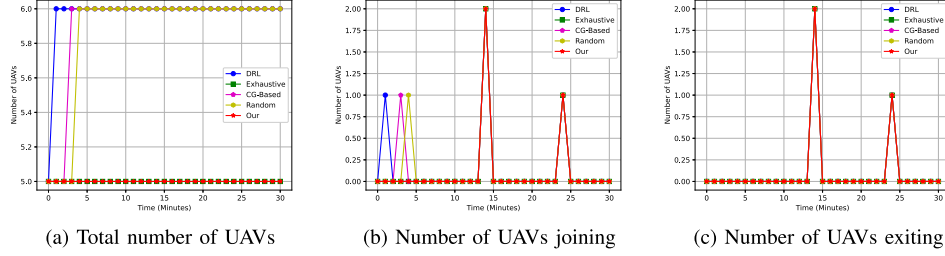


Fig. 7. Change in the number of UAVs in scenario II.

- 2) Exhaustive Search Method (Exhaustive): Exploring all possible combinations of channel allocation and resource scheduling to select the best-performing decision.
- 3) Channel Gain-based Greedy Strategy (CG-based): This strategy adopts an intuitive greedy approach, prioritizing communication resources for users with the highest channel gains.
- 4) Random Allocation Strategy (Random): The random allocation strategy assigns users to UAVs for communication and performs corresponding resource scheduling randomly.

C. Performance Evaluation

To systematically and comprehensively evaluate the suboptimality and convergence characteristics of the proposed method, we set scenario I (i.e., a 30-minute service cycle, with the UAV battery capacity for one hour of hovering flight). In this scenario, we specifically observed the long-term average queue length of

users ($\frac{\sum_{n=1}^N \sum_{t=1}^T Q_n^L(t)}{N \times T}$), the long-term average queue length of UAV-MEC servers ($\frac{\sum_{s=1}^S \sum_{t=1}^T Q_s^U(t)}{S \times T}$), the long-term average power consumption of UAV-MEC servers ($\frac{\sum_{s=1}^S \sum_{t=1}^T E_{mix}^s(t)}{S \times T}$), and the long-term average system power consumption ($\frac{\sum_{t=1}^T E_{mix}^{Total}(t)}{T}$) excluding UAV flight power consumption (Note: Once the system requires additional UAV support, this method will inevitably become sub-optimal, as the flight power consumption far exceeds the total computational offloading power consumption).

From Fig. 6(a), the Random and CG-Based methods initially show high levels, followed by a gradual decline. This phenomenon is attributed to their dynamic introduction of a new UAV into the network for offloading. In contrast, both our proposed method

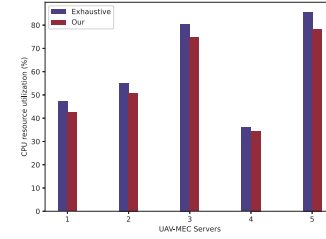


Fig. 8. CPU resource utilization comparison.

and the Exhaustive method demonstrate more stable queue length growth, with minimal differences between them. The DRL method introduces a new UAV for offloading from the start, but it exhibits significant long-term queue length growth. From Fig. 6(b), our proposed method and the Exhaustive method perform similarly, both maintaining stable states. From Fig. 6(c), we observe that the Random, CG-Based, and DRL methods are higher than the other two methods. From Fig. 6(d), our proposed method and the Exhaustive method yield similar results, indicating that our method effectively control overall system power consumption while maintaining service quality. In conclusion, our proposed method demonstrates energy consumption performance comparable to the Exhaustive method, while maintaining stability in both the user and UAV-MEC server queues, thereby validating its superiority and practicality.

To visually demonstrate the entry and exit behavior of UAVs, we set the UAV battery capacity to support 30 minutes of hovering flight (scenario II). From Fig. 7(a), our proposed method and the Exhaustive method consistently maintain a stable number of 5 UAVs. In contrast, the DRL, CG-Based, and Random methods increase the number of UAVs to 6 at different times—1 minute, 3 minutes, and 4 minutes after the service begins, respectively. From Fig. 7(b) and 7(c), we observed that at 14

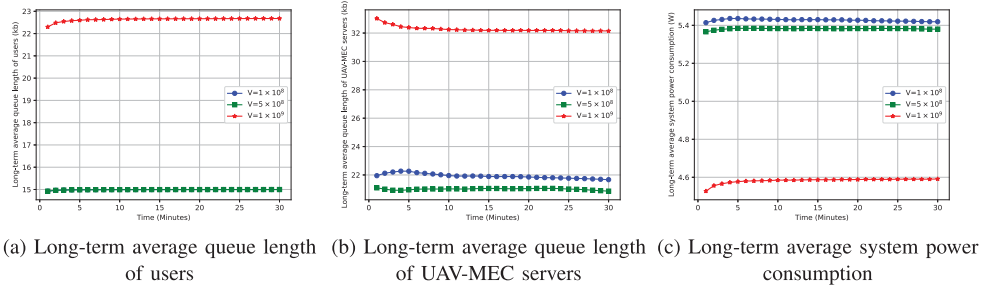


Fig. 9. Influence of control parameter V on long-term queues and long-term power consumption.

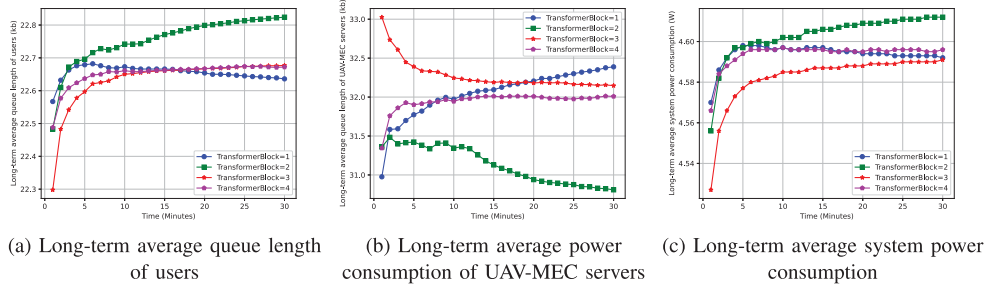


Fig. 10. Effect of TransformerBlock layers on long-term queue and long-term power consumption in TCAG.

minutes and 24 minutes, pairs of UAVs joined and exited, which was caused by the battery running low. This behavior demonstrates the effectiveness of the microservice management component.

D. Comparison With the Overhead of the Exhaustive

To further demonstrate the superiority of the proposed method, we compared the CPU resource utilization of the Exhaustive method during system operation. The simulation results are shown in Fig. 8. The proposed method exhibits lower CPU resource overhead on all five UAV-MEC servers compared to the Exhaustive method. Specifically, the difference is more pronounced on UAV-MEC servers 3 and 5 (About 5%). It is worth noting that, under the current settings, the computational complexity of the proposed method for each UAV-MEC server is $O(kI_{\max}(N_i + \log_2(N_i/\eta)))$, where the parameter k is set to 16. In contrast, the computational complexity of the Exhaustive method exhibits significantly different characteristics. Since the maximum number of connections for each UAV-MEC server is limited to 2, in a scenario where the number of users is N_i , the parameter k of the Exhaustive method can be expressed as $k = C_{N_i}^2$. This characteristic causes the computational complexity of the Exhaustive method to increase rapidly as the number of users grows. When the above conditions are determined, calculations show that when $N_i = 6$, the computational complexity of both the proposed method and the Exhaustive method become numerically similar. However, when the Exhaustive method is applied to larger-scale rescue scenarios, with more user connection requirements and wider user distribution, its computational complexity increases dramatically, making the consumption of computing resources

unsustainable. In summary, our method is more efficient in resource utilization, reducing the CPU burden under the same conditions and improving the overall operational efficiency of the system.

E. Influence Analysis of Important Parameters

1) *Effect of the Parameter V* : In Fig. 9, we investigate the impact of different sizes of the parameter V on various system performance metrics, with all other settings remaining consistent with the conditions of scenario I. From Fig. 9(a) and 9(b), we observe that smaller values of V have a greater impact on the queue, stabilizing the queue length at a lower level. However, as seen in Fig. 9(c), larger values of V place more weight on power consumption, resulting in lower power consumption.

2) *Effect of TransformerBlock Layers*: In Fig. 10, we investigate the impact of the number of TransformerBlock layers on queue and power consumption. From Fig. 10(a) and 10(b), it is evident that a smaller number of TransformerBlock layers has opposite effects on the two queues. In contrast, when the number of TransformerBlock layers is appropriate, the two queues remain relatively stable. From Fig. 10(c), An appropriate number of TransformerBlock layers has a positive effect on reducing system power consumption. Therefore, it is essential to find a balance between model complexity and system power consumption during the design process. Additionally, we analyzed the effect of the number of TransformerBlock layers on inference speed, and the results are shown in Fig. 11. As the number of layers increases, the average inference time of TCAG gradually increases. Therefore, in practical applications, it is necessary to

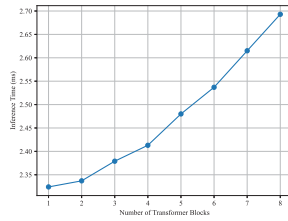


Fig. 11. Effect of TransformerBlock layers on inference time.

balance model complexity and inference speed to ensure efficient inference while improving model performance.

VI. CONCLUSION

In response to the urgent needs of emergency medical rescue after natural disasters, we proposed a UAV-assisted microservice MEC architecture. This architecture leverages the flexibility and mobility of UAVs, the modularity and rapid scalability of the microservice framework, and the efficiency of lightweight neural network models. It aims to provide stable, efficient, real-time computing and data support for rescue operations in scenarios where network and power infrastructure are damaged by natural disasters. The architecture ensures long-term stability through mechanisms for self-organizing networks and the automatic management of UAVs joining and exiting. Additionally, the TBRM method effectively balances data offloading, energy consumption, and latency, thereby maximizing the architecture's service time. Furthermore, we design four microservices to manage the full UAV lifecycle and introduce dual digital signature certificates to enhance system security. Extensive simulation results demonstrate the effectiveness of the proposed method. We also provide recommendations for key parameter settings, which further verify its feasibility in real rescue environments. In summary, this study provides a technological support solution for post-disaster emergency medical rescue, with both theoretical significance and practical value.

REFERENCES

- [1] C. Luo, J. Zhang, X. Cheng, Y. Hong, Z. Chen, and X. Xing, "Computation off-loading in resource-constrained edge computing systems based on deep reinforcement learning," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 109–122, Jan. 2024.
- [2] H. Lin, L. Yang, H. Guo, and J. Cao, "Decentralized task offloading in edge computing: An offline-to-online reinforcement learning approach," *IEEE Trans. Comput.*, vol. 73, no. 6, pp. 1603–1615, Jun. 2024.
- [3] D. Zhang, Z. Zhang, J. Zhang, T. Zhang, L. Zhang, and H. Chen, "Uav-assisted task offloading system using dung beetle optimization algorithm & deep reinforcement learning," *Ad Hoc Netw.*, vol. 156, 2024, Art. no. 103434. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1570870524000453>
- [4] H. Sun, X. Zhang, B. Zhang, K. Sha, and W. Shi, "Optimal task offloading and trajectory planning algorithms for collaborative video analytics with UAV-assisted edge in disaster rescue," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 6811–6828, May 2024.
- [5] H. Zeng, Z. Zhu, Y. Wang, Z. Xiang, and H. Gao, "Periodic collaboration and real-time dispatch using an actor-critic framework for UAV movement in mobile edge computing," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 2125–2126, Jun. 2024.
- [6] H. Wang, H. Xu, H. Huang, M. Chen, and S. Chen, "Robust task offloading in dynamic edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 500–514, Jan. 2023.
- [7] F. Li, J. Luo, Y. Qiao, and Y. Li, "Joint uav deployment and task offloading scheme for multi-UAV-assisted edge computing," *Drones*, vol. 7, no. 5, p. 284, 2023. [Online]. Available: <https://www.mdpi.com/2504-446X/7/5/284>
- [8] K. Cheng, X. Fang, and X. Wang, "Energy efficient edge computing and data compression collaboration scheme for UAV-assisted network," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16395–16408, Dec. 2023.
- [9] B. Xu, Z. Kuang, J. Gao, L. Zhao, and C. Wu, "Joint offloading decision and trajectory design for uav-enabled edge computing with task dependency," *IEEE Trans. Wireless Commun.*, vol. 22, no. 8, pp. 5043–5055, Aug. 2023.
- [10] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: 10.1145/3065386.
- [11] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [13] A. Howard et al., "Searching for mobilenetv3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1314–1324.
- [14] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet v2: Practical guidelines for efficient cnn architecture design," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 116–131.
- [15] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, Long Beach, CA, USA: Curran Associates, Inc., 2017.
- [16] K. Cheng et al., "ProScale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 4, pp. 1294–1312, Apr. 2023.
- [17] Y. Zhao, J. Wang, and B. Li, "A deep reinforcement learning approach to online microservice deployment in mobile edge computing," in *Proc. Int. Conf. Service-Oriented Comput.*, Cham: Springer, 2023, pp. 127–142.
- [18] M. D. Hossain et al., "The role of microservice approach in edge computing: Opportunities, challenges, and research directions," *ICT Exp.*, vol. 9, no. 6, pp. 1162–1182, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405959523000760>
- [19] F. Liu, Y. Guo, Z. Cai, N. Xiao, and Z. Zhao, "Edge-enabled disaster rescue: A case study of searching for missing people," vol. 10, no. 6, pp. 1–21, Dec. 2019. [Online]. Available: <https://doi.org/10.1145/3331146>
- [20] S. Mao, S. He, and J. Wu, "Joint UAV position optimization and resource scheduling in space-air-ground integrated networks with mixed cloud-edge computing," *IEEE Syst. J.*, vol. 15, no. 3, pp. 3992–4002, Sep. 2021.
- [21] S. H. Alsamhi et al., "UAV computing-assisted search and rescue mission framework for disaster and harsh environment mitigation," *Drones*, vol. 6, no. 7, p. 154, 2022. [Online]. Available: <https://www.mdpi.com/2504-446X/6/7/154>
- [22] G. Faraci, S. A. Rizzo, and G. Schembra, "Green edge intelligence for smart management of a fanet in disaster-recovery scenarios," *IEEE Trans. Veh. Technol.*, vol. 72, no. 3, pp. 3819–3831, Mar. 2023.
- [23] H. Zhang et al., "Decentralized and fault-tolerant task offloading for enabling network edge intelligence," *IEEE Syst. J.*, vol. 18, no. 2, pp. 1459–1470, Jun. 2024.
- [24] B. Cen et al., "A configuration method of computing resources for microservice-based edge computing apparatus in smart distribution transformer area," *Int. J. Electr. Power Energy Syst.*, vol. 138, 2022, Art. no. 107935. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0142061521011455>
- [25] F. Guo, B. Tang, and M. Tang, "Joint optimization of delay and cost for microservice composition in mobile edge computing," *World Wide Web*, vol. 25, no. 5, pp. 2019–2047, 2022.
- [26] W. Lv et al., "Microservice deployment in edge computing based on deep q learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2968–2978, Nov. 2022.
- [27] F. Guo, B. Tang, M. Tang, and W. Liang, "Deep reinforcement learning-based microservice selection in mobile edge computing," *Cluster Comput.*, vol. 26, no. 2, pp. 1319–1335, 2023.
- [28] B. Tang, F. Guo, B. Cao, M. Tang, and K. Li, "Cost-aware deployment of microservices for IoT applications in mobile edge computing environment," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3119–3134, Sep. 2023.
- [29] K. Ray, A. Banerjee, and N. C. Narendra, "Learning-based microservice placement and migration for multi-access edge computing," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1969–1982, Apr. 2024.

- [30] L. T. Hoang, C. T. Nguyen, and A. T. Pham, "Deep reinforcement learning-based online resource management for UAV-assisted edge computing with dual connectivity," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 2761–2776, Dec. 2023.
- [31] Y. Zhang, C. Chen, L. Liu, D. Lan, H. Jiang, and S. Wan, "Aerial edge computing on orbit: A task offloading and allocation scheme," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 1, pp. 275–285, Jan./Feb. 2023.
- [32] Y. Ding et al., "Online edge learning offloading and resource management for UAV-assisted MEC secure communications," *IEEE J. Sel. Topics Signal Process.*, vol. 17, no. 1, pp. 54–65, Jan. 2023.
- [33] M. K. Hasan et al., "Federated learning for computational offloading and resource management of vehicular edge computing in 6g-v2x network," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 3827–3847, Feb. 2024.
- [34] H. Zhang, J. Wang, H. Zhang, and C. Bu, "Security computing resource allocation based on deep reinforcement learning in serverless multi-cloud edge computing," *Future Gener. Comput. Syst.*, vol. 151, pp. 152–161, 2024.
- [35] Zhang, Y. *Mobile Edge Computing*. Berlin, Germany: Springer Nature, 2022.
- [36] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint computation offloading and user association in multi-task mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12 313–12 325, 2018.
- [37] S. Jeong, O. Simeone, and J. Kang, "Mobile edge computing via a UAV-mounted cloudlet: Optimization of bit allocation and path planning," *IEEE Trans. Veh. Technol.*, vol. 67, no. 3, pp. 2049–2063, Mar. 2018.
- [38] J. Zhang et al., "Stochastic computation offloading and trajectory scheduling for UAV-assisted mobile edge computing," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3688–3699, Apr. 2019.
- [39] H. V. Abeywickrama, B. A. Jayawickrama, Y. He, and E. Dutkiewicz, "Empirical power consumption model for UAVs," in *Proc. IEEE 88th Veh. Technol. Conf. (VTC-Fall)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 1–5.



Ji Li received the B.S. and M.S. degrees from Taiyuan University of Science and Technology, China. He is currently working toward the Ph.D. degree with the Northeastern University, Shenyang, China. His research interests include social network, edge computing, and computer vision.



ing, health care, infectious diseases informatics, etc.

Qiang He (Associate Member, IEEE) received the Ph.D. degree in computer application technology from the Northeastern University, Shenyang, China, in 2020. He also worked with the School of Computer Science and Technology, Nanyang Technical University, Singapore as a visiting Ph.D. Researcher from 2018 to 2019. Currently, he is an Associated Professor with the College of Medicine and Biological Information Engineering, Northeastern University, Shenyang, China. His research interests include machine learning, social network analytic, data mining, health care, infectious diseases informatics, etc.



Xingwei Wang received the B.S., M.S., and Ph.D. degrees in computer science from the Northeastern University, Shenyang, China, in 1989, 1992, and 1998, respectively. Currently, he is a Professor with the College of Computer Science and Engineering, Northeastern University, Shenyang, China. His research interests include cloud computing and future Internet.



Ammar Hawbani received the B.S., M.S., and Ph.D. degrees in computer software and theory from the University of Science and Technology of China (USTC), Hefei, China, in 2009, 2012, and 2016, respectively. He is an Associate Professor of networking and communication algorithms with the School of Computer Science and Technology, USTC. From 2016 to 2019, he worked as a Postdoctoral Researcher with the School of Computer Science and Technology, USTC.



Keping Yu (Senior Member, IEEE) received the M.E. and Ph.D. degrees from the Graduate School of Global Information and Telecommunication Studies, Waseda University, Japan, in 2012 and 2016, respectively. He was a Research Associate, a Junior Researcher, a Researcher with the Global Information and Telecommunication Institute, Waseda University, from 2015 to 2019, 2019 to 2020, and 2020 to 2022, respectively. Currently, he is an Associate Professor, the Vice Director of Institute of Integrated Science and Technology, and the Director of the Network Intelligence and Security Laboratory (YU Lab), Hosei University, Japan.



Yuanguo Bi (Member, IEEE) received the Ph.D. degree in computer science and technology from the Northeastern University, Shenyang, China, in 2010. He was a visiting Ph.D. student with the BroadBand Communications Research (BBCR) Lab, Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON, Canada, from 2007 to 2009. Currently, he is a Professor with the School of Computer Science and Engineering, Northeastern University, China.



Liang Zhao received the Ph.D. degree from the School of Computing, Edinburgh Napier University, in 2011. He is a Professor with Shenyang Aerospace University, China. Before joining Shenyang Aerospace University, he worked as an Associate Senior Researcher with Hitachi (China) Research and Development Corporation, from 2012 to 2014. He is also a JSPS Invitational Fellow (2023).